Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering & Research, Ravet, Pune

An Autonomous Institute | NBA Accredited | NAAC A++ Accredited | ISO 21001:2018 Certified

IQAC PCCOER

MINI PROJECT (MP)

Course Code: MDM-331-ETC

WORKBOOK | ACADEMIC YEAR 2026–27 | TERM I

PROJECT TITLE

AI-Based Hand Gesture Recognition and Touchless Computer Control System

AREA OF PROJECT

Machine Learning, Artificial Intelligence, Computer Vision and Human-Computer Interaction

Division: _____

Name of Faculty/Mentor: _____

Student details are intentionally left blank for completion.

CERTIFICATE

This is to certify that the student(s) named below have completed the term work and practical work in the subject Mini Project (MP) in the Department of Electronics & Telecommunication Engineering for the academic year 2026–27, on the project titled:

AI-Based Hand Gesture Recognition and Touchless Computer Control System

1. _____

2. _____

3. _____

4. _____

5. _____

6. _____

Subject Teacher **Head of Department** **Principal**

Date: ____ / ____ / _____

GENERAL RULES & REGULATIONS

- Handle the workbook carefully.
- All students must enter correct information in the workbook.
- All entries must be verified by the concerned Supervisor/Mentor.
- Activities should be completed according to the schedule given by the Supervisor/Mentor.

CONTENT

Sr. No.	Title	Page
1	Vision and Mission statements of Institute and Department	4
2	Program Outcomes and Course Outcomes	5–6
3	Selection of Project / Problem	7
4	Recommended Guidelines and Group Information	8
5	Abstract and Introduction	9–10
6	Literature Survey	11–12
7	Problem Statement, Objectives and Results	13–14
8	Block Diagram and System Workflow	15–18
9	Software Design and Implementation	19
10	Testing, Evaluation and Limitations	20
11	Conclusion and Future Scope	21
12	References and Required Hardware/Software	22

VISION AND MISSION STATEMENTS

Vision and Mission of the Institute

Vision: To be a globally recognized Institute of technological education and research for the holistic development of aspirants, through excellence in education, innovation and collaborations to fulfill the expectations of all stakeholders.

Mission

- To design and deliver state-of-the-art knowledge through experiential learning based on changing needs of industry and society worldwide, to ensure the employability and employment of each aspirant.
- To enhance collaborative partnerships between industry and institute at national and international levels for technological transfer and societal, ethical and economic development.
- To achieve and sustain the institute's position as a recognized leader in technical and technological education.

Vision and Mission of the Department

Vision: To be a globally recognized centre of excellence in Electronics and Telecommunication Engineering through quality education, innovation and research.

Mission

- To impart cutting-edge knowledge and practical skills through innovative and experiential learning.
- To foster research, innovation and collaboration with industry and academia.
- To develop competent, ethical and industry-ready professionals through quality teaching and emerging technologies.

PROGRAM EDUCATIONAL OBJECTIVES AND OUTCOMES

Program Educational Objectives

- Provide a learning environment and hands-on training in Electronics and Telecommunication Engineering.
- Inculcate research and innovation capabilities in E&TC and allied interdisciplinary domains.
- Enhance professional competencies through industry engagement, research, innovation and lifelong learning.

Program Specific Outcomes

- Build and test electronics and telecommunication systems using software and hardware tools for real-life problems.
- Develop and support modern electronic systems.

Relevant Program Outcomes

- **Engineering knowledge:** apply mathematics, science, computing and engineering fundamentals.
- **Problem analysis:** identify, formulate and analyze engineering problems using research literature.
- **Design/development of solutions:** design systems and processes to meet identified needs.
- **Conduct investigations:** use experiments, modelling, analysis and interpretation of data.
- **Engineering tool usage:** apply modern engineering and IT tools while recognizing their limitations.
- **Communication and teamwork:** communicate effectively and work collaboratively.
- **Lifelong learning:** adapt to emerging technologies and technological change.

COURSE OBJECTIVES AND COURSE OUTCOMES

Course Objectives

- Introduce fundamentals of Artificial Intelligence, Machine Learning and data-driven systems.
- Develop ability to preprocess, analyze and visualize engineering datasets.
- Enable implementation of Machine Learning algorithms using Python-based tools.
- Introduce emerging technologies such as Edge AI, TinyML, AIoT and Generative AI.
- Prepare students for industry-oriented AI projects, internships, research and multidisciplinary applications.

Course Outcomes

Code	Course Outcome
MDM-331.1	Explain Machine Learning fundamentals, workflow and AI applications in E&TC.
MDM-331.2	Apply data preprocessing, visualization and exploratory analysis using Python libraries.
MDM-331.3	Implement supervised Machine Learning algorithms for prediction and classification.
MDM-331.4	Apply unsupervised learning techniques for clustering engineering datasets.
MDM-331.5	Develop AI-based mini-projects using modern tools, real-world datasets and emerging technologies.

Project alignment: This project primarily demonstrates computer vision, real-time hand-landmark inference, rule-based gesture classification, Python-based implementation and human-computer interaction. It maps mainly to MDM-331.1, MDM-331.2 and MDM-331.5, and its future scope (a learned gesture classifier) extends to MDM-331.3.

SELECTION OF PROJECT / PROBLEM

Selected Project Title

AI-Based Hand Gesture Recognition and Touchless Computer Control System

Project Area

Machine Learning, Computer Vision, Image Processing and Human-Computer Interaction.

Background and Motivation

Conventional computer input devices such as the keyboard, mouse and touchpad require physical contact. In situations such as classroom presentations, shared or public computers, sterile laboratory and clinical environments, and assistive use by people with limited mobility, a contact-free method of controlling a computer is useful. Computer vision allows a standard camera to observe the hand and interpret its pose as an input command.

The project uses a standard webcam and a pretrained hand-landmark detection model (MediaPipe Hands) to locate 21 key points of the hand in real time. Rule-based logic then interprets the finger configuration and fingertip distances as gestures, which are mapped to cursor movement, left click, right click and scrolling on the host computer.

Reason for Selection

The project gives practical experience in artificial intelligence, computer vision, real-time video processing, Python programming and human-computer interaction. It is relevant to smart classrooms, touchless public kiosks, assistive technology and interactive systems, and requires no hardware beyond a laptop or desktop computer.

Scope

The proposed version is software-only. A laptop or desktop computer with a webcam performs capture, landmark detection, gesture classification and system control. The default pretrained hand-landmark model is used as the implementation baseline; a learned gesture classifier may be added as future work. No external electronic hardware is required.

RECOMMENDED GUIDELINES AND GROUP INFORMATION

Group Information

Class: _____ Batch: _____

Roll No.	PRN No.	Name of Student	Mobile No.

Name of Faculty: _____
E-mail: _____
Mobile No.: _____

Recommended Project Phases

Phase	Activity	Expected output
1	Problem selection and literature survey	Finalized project scope and references
2	Environment setup and library selection	Python environment with MediaPipe, OpenCV, PyAutoGUI
3	Webcam capture and hand-landmark detection	Live video with 21 landmarks overlaid
4	Gesture recognition logic	Finger-state and pinch-based gesture classification
5	Cursor mapping and system control	Cursor movement, click and scroll operations
6	Testing and evaluation	Test observations, frame rate and gesture accuracy
7	Documentation and presentation	Final report and demonstration

ABSTRACT

The project, “AI-Based Hand Gesture Recognition and Touchless Computer Control System”, presents a software-based computer vision system that allows a user to operate a computer without touching a mouse or touchpad. The system captures live video from a standard webcam, detects the user’s hand using a pretrained hand-landmark model, and extracts 21 landmark points that describe the pose of the hand and fingers.

A rule-based gesture recognition module analyses which fingers are extended and the distances between selected fingertips to identify gestures such as pointing, pinching, two-finger scrolling, open palm and closed fist. Each gesture is mapped to a computer action: moving the cursor, left click, right click, scrolling or pausing control. The index fingertip position is mapped from the camera frame to the screen and smoothed to reduce jitter.

Python is the programming language; OpenCV handles video capture and display, MediaPipe provides hand landmark detection, NumPy supports numerical computation and PyAutoGUI sends cursor and click commands to the operating system. The prototype focuses on model integration, gesture logic, system control and testing rather than claiming that a new model has been trained from scratch. Recognition quality depends on lighting, background, camera quality, hand distance and the chosen thresholds.

Keywords: Artificial Intelligence, Computer Vision, Hand Gesture Recognition, MediaPipe, Human-Computer Interaction, Touchless Control, Python, OpenCV.

INTRODUCTION

1. Background

Human-computer interaction (HCI) studies how people communicate with machines. The keyboard and mouse remain the dominant interfaces, but they need physical contact and a flat surface. Vision-based interfaces use cameras to understand body movements and offer a more natural, contact-free alternative.

2. Hand Gesture Recognition

Hand gesture recognition is the task of interpreting the shape and movement of the hand as a meaningful command. Gestures may be static (a fixed hand pose such as a fist) or dynamic (a movement over time such as a swipe). This project focuses on static poses combined with continuous fingertip tracking.

3. Hand-Landmark Detection

MediaPipe Hands is a lightweight machine-learning solution that first locates the palm and then predicts 21 three-dimensional landmarks of the hand, together with a handedness label (left or right). Because it runs in real time on a normal CPU, it is suitable for interactive applications. The landmark positions are the input to the gesture logic of this project.

4. Proposed System

The webcam supplies video frames. Each frame is mirrored and passed to the hand-landmark model. The gesture module determines finger states and pinch distances, classifies the gesture, and the action module converts it into a cursor, click or scroll command. A preview window shows the landmarks, the recognised gesture and the frame rate.

5. Scope and Limitations

The current implementation is intended for educational demonstration and works with a single hand. It does not identify the person. It may misclassify gestures under poor lighting, cluttered backgrounds, motion blur or when the hand is partly outside the frame, and should not be treated as a safety-critical interface.

INITIAL SURVEY FOR FINALIZATION OF TITLE

The following sources provide the background for hand tracking, gesture-based interaction and the software tools used in the proposed system.

Ref.	Source	Main contribution	Relevance
[1]	Zhang et al., 2020 – MediaPipe Hands	On-device real-time hand tracking predicting 21 hand landmarks.	Core landmark detection model used in this project.
[2]	Lugaresi et al., 2019 – MediaPipe	Framework for building cross-platform perception pipelines.	Underlying framework of the hand-tracking solution.
[3]	Rautaray and Agrawal, 2015	Survey of vision-based hand gesture recognition for HCI.	Background on gesture types and challenges.
[4]	OpenCV Documentation	Video capture, image processing and drawing utilities.	Webcam handling and on-screen visualization.
[5]	PyAutoGUI Documentation	Programmatic control of mouse, keyboard and screen.	Sends cursor, click and scroll commands.

Summary of Literature Survey

The reviewed work establishes vision-based hand tracking as a practical route to touchless human-computer interaction. Landmark-based models such as MediaPipe Hands remove the need for gloves or special sensors, and open-source Python libraries make it possible to connect landmark detection with gesture logic and operating-system control in a single, reproducible application.

LITERATURE SURVEY – EXISTING SYSTEMS AND RESEARCH GAP

Existing Approaches

- **Physical input devices:** mouse, keyboard and touchpad are accurate and familiar but require contact.
- **Wearable sensor gloves:** flex and motion sensors give precise finger data but need worn hardware and calibration.
- **Dedicated depth or infrared sensors:** specialised cameras track hands accurately but add cost and extra devices.
- **Classical image processing:** skin-colour segmentation and contour analysis work in controlled conditions but are sensitive to lighting and background.
- **Deep-learning landmark detection:** pretrained models predict hand key points from ordinary RGB video and generalise across users and backgrounds.

Limitations

- Wearables and dedicated sensors increase cost and reduce convenience.
- Colour-segmentation methods need extensive tuning for each environment.
- Pretrained landmark models can lose the hand under low light, blur or occlusion.
- Gesture-to-action mapping must be designed carefully to avoid accidental clicks and cursor jitter.

Gap Addressed by This Project

This project provides a student-level, reproducible workflow that uses only a standard webcam and open-source libraries. It combines pretrained hand-landmark detection with transparent gesture rules, cursor smoothing and safety features such as a pause gesture, and demonstrates how a machine-learning model can be integrated into a usable system that controls a real computer.

PROBLEM STATEMENT AND OBJECTIVES

Problem Statement

Operating a computer through physical input devices is not always convenient or hygienic, and is difficult for some users. A software-based system is therefore required to detect the user's hand through an ordinary webcam, recognise a set of hand gestures in real time and translate them into cursor and click operations. The proposed system addresses this need using a pretrained hand-landmark model and a Python-based control module.

Objectives of Project

1. To study the fundamentals of computer vision, machine learning and hand gesture recognition.
2. To develop a Python application that captures live video and detects hand landmarks in real time.
3. To design gesture-recognition rules based on finger states and fingertip distances.
4. To map recognised gestures to cursor movement, left click, right click and scrolling.
5. To reduce cursor jitter and accidental actions using smoothing, cooldown and a pause gesture.
6. To test the system under different conditions and document its limitations and performance.

Expected Application Areas

- Touchless control of presentations in classrooms and seminar halls.
- Hygienic interaction with shared or public kiosks and sterile environments.
- Assistive technology for users with limited mobility.
- Interactive displays, gaming and AR/VR research prototypes.

RESULTS AS PER OBJECTIVES

Status note: The table below lists expected outcomes and verification methods. Actual results, screenshots and performance values must be entered after the application has been run and tested. No experimental accuracy figures are claimed in this report.

No.	Objective	Expected outcome	Verification method
1	Hand detection	21 landmarks displayed on the live hand.	Observe preview window under normal lighting.
2	Cursor control	Cursor follows the index fingertip smoothly.	Move across the full screen and check coverage.
3	Click operations	Pinch gestures trigger left and right clicks.	Click on test targets and count correct actions.
4	Scrolling	Two-finger gesture scrolls a page up and down.	Scroll a long document in both directions.
5	Safety control	Fist gesture pauses all actions.	Verify no cursor or click events while paused.
6	Performance evaluation	Frame rate and gesture accuracy recorded.	Repeat each gesture over a set number of trials.

Evaluation Metrics

- **Gesture accuracy** = correctly recognised trials / total trials.
- **Precision** = TP / (TP + FP) and **Recall** = TP / (TP + FN), computed per gesture from a confusion matrix.
- **F1-score** = 2 × Precision × Recall / (Precision + Recall).
- **Frame rate (FPS)** = number of frames processed per second; indicates real-time suitability.
- **Latency** = delay between a gesture and the resulting computer action.
- **False trigger rate** = unintended clicks or scrolls per minute of use.

FIGURE / BLOCK DIAGRAM / FLOW CHART

System Block Diagram

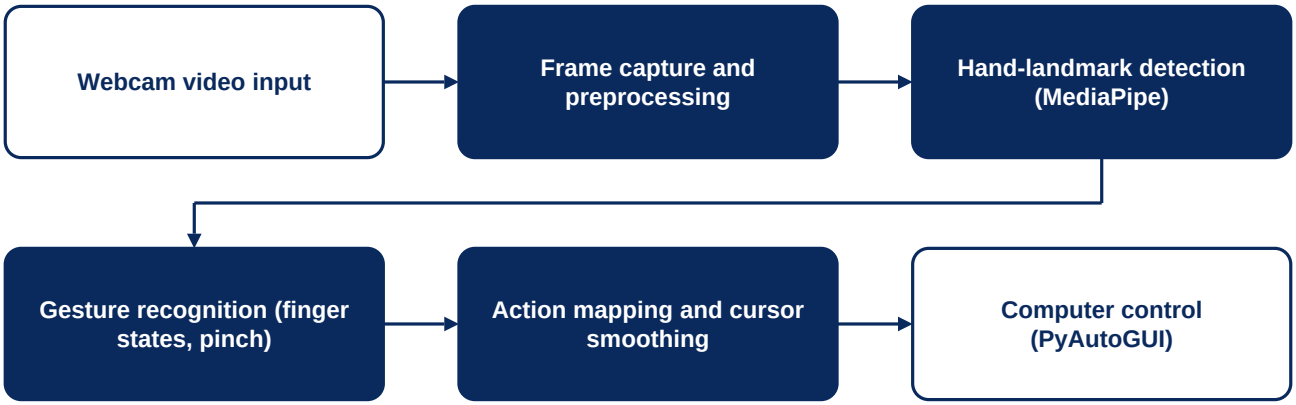


Fig. 1: System block diagram of the AI-based hand gesture control system.

Working Principle

- The webcam streams live video to the application, and each frame is mirrored for natural interaction.
- The frame is converted from BGR to RGB and passed to the pretrained hand-landmark model.
- The model returns 21 landmark coordinates and the handedness of the detected hand.
- The gesture module computes finger states and fingertip distances and classifies the gesture.
- The action module maps the gesture to a cursor, click or scroll command and sends it to the operating system.
- A preview window displays landmarks, the recognised gesture name and the frame rate for user feedback.

SYSTEM WORKFLOW

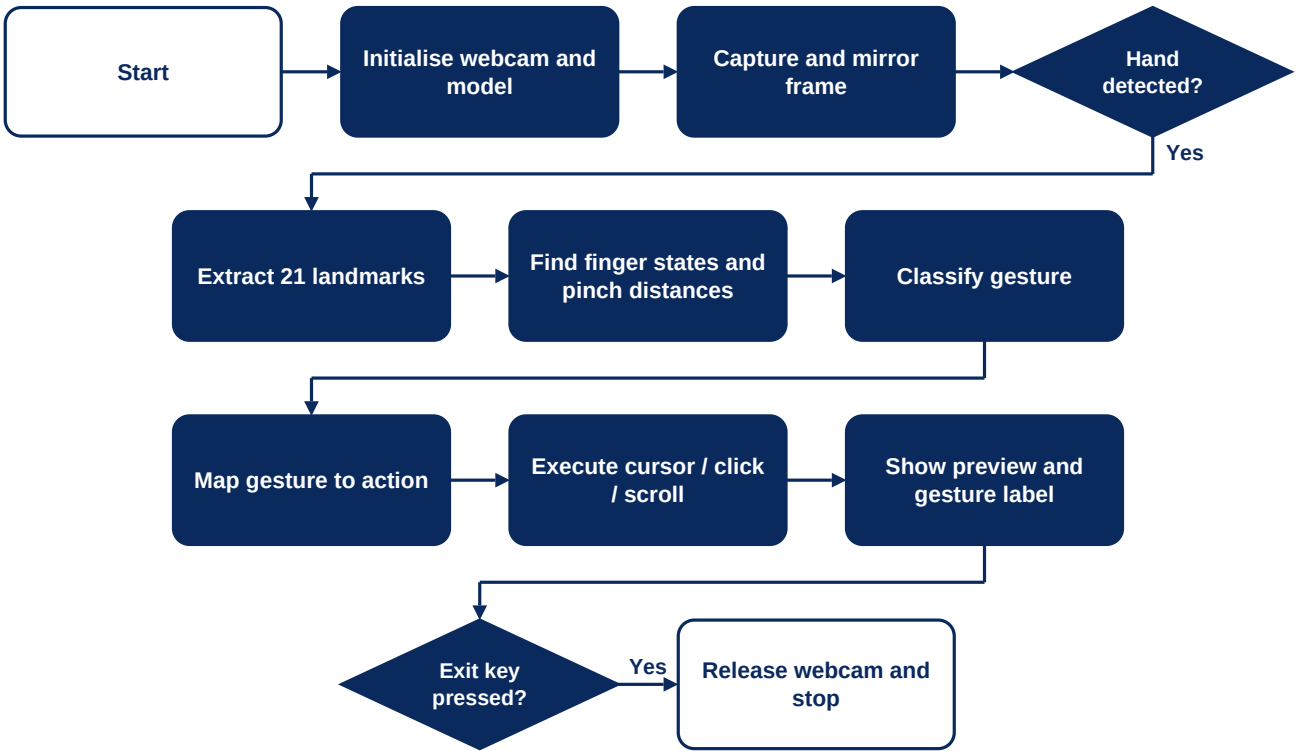


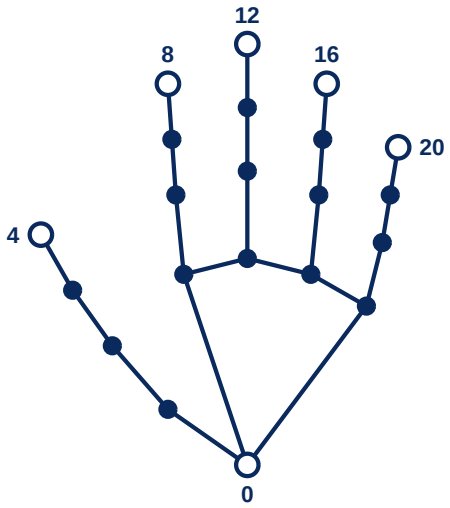
Fig. 2: Real-time processing and control workflow. If no hand is detected, or the exit key is not pressed, the loop returns to frame capture.

Main Modules

Module	Description
Input module	Captures webcam frames and mirrors them.
Landmark module	Runs the pretrained hand-landmark model and returns 21 landmarks.
Gesture module	Determines finger states and pinch distances and classifies the gesture.
Control module	Maps the fingertip to screen coordinates, smooths motion and issues cursor, click and scroll commands.
Interface module	Draws landmarks, gesture label and FPS on the preview and handles exit and pause.

GESTURE RECOGNITION DESIGN

Hand Landmark Model



ID	Landmark
0	Wrist
1–4	Thumb (CMC, MCP, IP, tip)
5–8	Index finger (MCP, PIP, DIP, tip)
9–12	Middle finger
13–16	Ring finger
17–20	Little finger

Fig. 3: The 21 hand landmarks predicted by the model (key fingertip points numbered).

Gesture Rules and Actions

Gesture	Detection rule	Action
Index finger up	Only the index finger is extended.	Move cursor
Index–thumb pinch	Distance between landmarks 4 and 8 below the pinch threshold.	Left click
Middle–thumb pinch	Distance between landmarks 4 and 12 below the pinch threshold.	Right click
Index + middle up	Index and middle fingers extended, others folded.	Scroll up / down
Open palm	All five fingers extended.	Idle (no action)
Closed fist	All fingers folded.	Pause control

Pinch distances are normalised by the palm size (distance between landmarks 0 and 9) so that the rules work at different distances from the camera. When several rules match, the priority order is pause, then pinch, then scroll, then move.

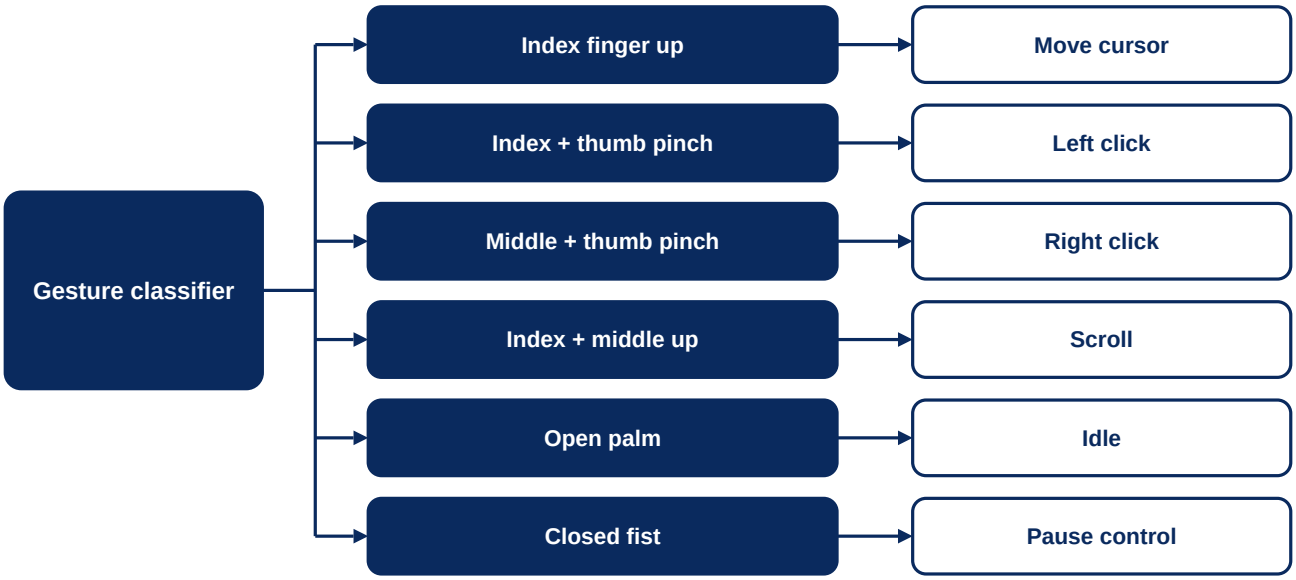


Fig. 4: Mapping of recognised gestures to computer actions.

SOFTWARE DESIGN AND IMPLEMENTATION

Technology Stack

Technology	Purpose
Python	Main programming language.
MediaPipe Hands	Pretrained hand detection and 21-point landmark model.
OpenCV	Webcam capture, frame conversion and on-screen drawing.
NumPy	Distance calculation and coordinate interpolation.
PyAutoGUI	Moves the cursor, clicks and scrolls through the operating system.

Implementation Steps

1. Install Python and the required libraries (opencv-python, mediapipe, pyautogui, numpy).
2. Open the webcam through OpenCV and set the frame resolution.
3. Initialise the hand-landmark model with detection and tracking confidence thresholds.
4. For each frame, mirror the image, convert it to RGB and run the model.
5. Read the 21 landmarks and convert them to pixel coordinates.
6. Determine finger states (a finger is up when its tip is above its PIP joint) and pinch distances.
7. Classify the gesture using the priority rules and apply a short cooldown to click events.
8. Map the index fingertip to screen coordinates, smooth it and move the cursor.
9. Display landmarks, gesture name and FPS, and exit cleanly on the exit key.

Cursor Mapping and Smoothing

```
x_screen = interp(x_cam, [margin, cam_width - margin], [0, screen_width])
y_screen = interp(y_cam, [margin, cam_height - margin], [0, screen_height])
position = previous + (target - previous) / k
```

A frame margin lets the user reach screen edges without extending the hand to the border of the camera view. The smoothing factor k reduces jitter; larger values give steadier but slower cursor response and must be tuned experimentally.

Model Information

The prototype uses the pretrained MediaPipe hand-landmark model for detection; it is not a model trained from scratch by the project team. Gesture classification is rule-based and transparent, which makes thresholds easy to explain and tune.

TESTING, EVALUATION AND LIMITATIONS

Test Plan

Test	Input / condition	Expected behaviour	Status
1	Open palm in good lighting	Hand and 21 landmarks detected; no action.	To be filled after testing
2	Index finger moved across the view	Cursor follows smoothly across the screen.	To be filled after testing
3	Index–thumb pinch	Single left click per pinch.	To be filled after testing
4	Middle–thumb pinch	Single right click per pinch.	To be filled after testing
5	Two fingers moved up and down	Page scrolls in the matching direction.	To be filled after testing
6	Closed fist	All actions paused.	To be filled after testing
7	Low-light or cluttered background	Detection may reduce; no false clicks.	To be filled after testing
8	Hand leaves the frame	Control stops without errors.	To be filled after testing

Limitations

- Recognition depends on lighting, background, camera quality and hand distance from the camera.
- Only one hand is supported in the current version.
- Rule-based thresholds may need re-tuning for different users and cameras.
- Frame rate depends on the computer hardware and camera resolution.
- Precise accuracy figures require repeated trials recorded under defined conditions.

Ethical, Privacy and Safety Considerations

- Video frames are processed locally and need not be stored; the system does not identify individuals.
- The camera should be used with the consent of the user and anyone in view.
- A pause gesture and the PyAutoGUI fail-safe (moving the cursor to a screen corner) allow the user to stop control immediately.
- The system should not be used for safety-critical operations.

CONCLUSION AND FUTURE SCOPE

Conclusion

The project presents a software-based touchless computer control system using Python, a pretrained hand-landmark model and computer vision libraries. It captures live webcam video, detects 21 hand landmarks, classifies a set of hand gestures and converts them into cursor movement, clicks and scrolling. The design is low-cost, requires only a standard webcam, and is easy to understand and extend.

The project provides practical exposure to real-time video processing, landmark-based inference, gesture logic design and human-computer interaction. Its reliability must be assessed through recorded tests, and the pretrained model should not be described as a model trained from scratch by the project team.

Applications

- Touchless control of presentations and smart classrooms.
- Hygienic interaction with public kiosks and sterile environments.
- Assistive computer access for users with limited mobility.
- Interactive displays, gaming and AR/VR prototypes.

Future Scope

- Train a supervised classifier (for example KNN, SVM or a small neural network) on landmark data to recognise more gestures.
- Add dynamic gestures such as swipes using temporal models.
- Support two hands, drag-and-drop and keyboard shortcuts.
- Add personal calibration of thresholds and automatic lighting adjustment.
- Evaluate precision, recall, F1-score, frame rate and latency on a recorded test set.
- Port the system to an embedded or edge platform such as Raspberry Pi.

REFERENCES AND REQUIRED HARDWARE / SOFTWARE

References (Websites / Books / Papers)

[1] F. Zhang et al., “MediaPipe Hands: On-device Real-time Hand Tracking,” arXiv:2006.10214, 2020. https://arxiv.org/abs/2006.10214

[2] C. Lugaresi et al., “MediaPipe: A Framework for Building Perception Pipelines,” arXiv:1906.08172, 2019. https://arxiv.org/abs/1906.08172

[3] S. S. Rautaray and A. Agrawal, “Vision based hand gesture recognition for human computer interaction: a survey,” Artificial Intelligence Review, vol. 43, pp. 1–54, 2015.

[4] MediaPipe Hand Landmarker Documentation. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker

[5] OpenCV Documentation. https://docs.opencv.org/

[6] PyAutoGUI Documentation. https://pyautogui.readthedocs.io/

[7] NumPy Documentation. https://numpy.org/doc/

[8] Python Documentation. https://docs.python.org/3/

Required Hardware

No.	Component	Purpose
1	Laptop / desktop computer	Runs Python, the landmark model and the control application.
2	Webcam or built-in camera	Captures live video of the user’s hand.
3	Internet connection (initial setup)	Installs packages and model files.

Required Software

No.	Software / Library	Purpose
1	Python 3.10–3.12	Main programming environment.
2	MediaPipe	Hand detection and landmark prediction.
3	OpenCV	Video capture and visualization.
4	NumPy	Numerical computation.
5	PyAutoGUI	Cursor, click and scroll control.
6	Visual Studio Code / IDE	Code editing and execution.

Signature of Faculty/Mentor: _____